

Estudo de Caso A1 — O Problema da Entrega Inteligente

Estruturas de Dados e Análise de Algoritmos

Prof. Alexandre Montanha

Questão 1

a)

Acredito que o problema da FastBite se encaixa na classe NP-Completo. O que me leva a pensar isso é que, ao misturar a decisão de qual entregador pega qual pedido com a decisão de qual ordem ele deve seguir para fazer as entregas, o número de combinações possíveis cresce de forma absurda conforme mais pedidos e entregadores entram no sistema. Dá pra verificar se uma solução é boa de forma rápida (só calcular o custo da rota e ver se as restrições foram cumpridas), o que encaixa na definição de NP. E como o problema se parece muito com o TSP, que é um problema NP-Completo clássico, faz sentido classificá-lo da mesma forma.

Lembrando as definições: P são problemas que conseguimos resolver em tempo polinomial. NP são problemas em que conseguimos verificar a solução rapidamente, mesmo que achar essa solução seja difícil. NP-Completo são os problemas mais difíceis dentro de NP, aqueles para os quais qualquer problema em NP pode ser reduzido.

b)

Pra mostrar a relação com o TSP, imagina um entregador com três pedidos pra pegar. Cada pedido tem um restaurante (ponto de coleta) e um cliente (ponto de entrega), então temos seis pontos no total. O entregador precisa passar por todos esses pontos, mas com a regra de que tem que pegar o pedido no restaurante antes de levar pro cliente.

Isso é muito parecido com o problema do caixeiro viajante, que precisa visitar todas as cidades passando pelo menor caminho possível. A diferença aqui é só essa restrição de ordem (coleta antes da entrega), mas a estrutura do problema é basicamente a mesma. Por isso dá pra dizer que o problema da FastBite pode ser transformado numa instância do TSP.

c)

Com 8 pedidos e 3 entregadores, a quantidade de combinações possíveis já é enorme. Só pra distribuir os 8 pedidos entre os 3 entregadores, temos em torno de 3 elevado a 8, ou seja, 6.561 formas diferentes de atribuição. Aí pra cada uma dessas distribuições, ainda precisamos calcular todas as ordens possíveis de visita para cada entregador.

Num cenário de pico com 50 pedidos e 20 entregadores, esse número passa de 10 elevado a 50 combinações possíveis. Pra ter uma ideia, um computador comum faz bilhões de operações por segundo, mas mesmo assim não terminaria de avaliar todas essas possibilidades nem em mil anos. Por isso a força bruta é completamente inviável na prática.

A complexidade assintótica desse tipo de abordagem fica em torno de $O(n! \times m^{\text{elevado a } n})$, o que confirma que o problema cresce de forma totalmente fora de controle conforme n e m aumentam.

Questão 2

a)

Usando os dados do cenário, o algoritmo funcionaria mais ou menos assim: o sistema pega o primeiro pedido da fila, que seria o P1, no restaurante na posição (0,2). Depois calcula a distância de cada entregador até esse restaurante. O E1 está em (1,1), então a distância Manhattan é 1 mais 1, que dá 2. O E2 está em (5,3), distância 5 mais 1, que dá 6. O E3 está em (7,6), distância 7 mais 4, que dá 11. O E1 é o mais próximo e tem capacidade disponível, então P1 vai pra ele.

O processo se repete pra P2, P3, P4 e P5, sempre atribuindo ao entregador disponível mais próximo do restaurante daquele pedido. Depois que todos os pedidos foram atribuídos, cada entregador organiza sua rota indo sempre ao ponto mais próximo de onde ele está no momento.

b)

Esse algoritmo é guloso porque ele toma a melhor decisão no momento em que está, sem pensar no que vai acontecer depois. A cada pedido, ele simplesmente olha quem está mais perto do restaurante agora e faz a atribuição. Não tem retrocesso, não tem planejamento de longo prazo. A escolha local é: pegar o entregador mais próximo do ponto de coleta naquele instante.

c)

Imagina que os entregadores E1 e E2 estão à mesma distância do restaurante de um pedido, mas o destino final desse pedido fica bem perto da posição atual de E2 e muito longe de E1. O algoritmo guloso, como só olha quem está mais perto do restaurante, pode acabar mandando E1 buscar esse pedido, mesmo que E2 fosse uma opção muito melhor no total. No final, E1 faz um percurso bem maior do que seria necessário, e a entrega acaba sendo mais lenta do que se tivesse sido pensada de forma global.

d)

A complexidade do algoritmo guloso é $O(n \times m)$, porque para cada um dos n pedidos, o sistema precisa comparar a distância até todos os m entregadores disponíveis. Na parte de montar a rota de cada entregador, se ele tem poucos pedidos, esse custo adicional é pequeno e não muda a complexidade dominante.

Questão 3

a)

Acredito que a Programação Dinâmica é aplicável ao problema de roteamento de um único entregador com k pedidos, já que esse tipo de problema tem as duas características que a PD precisa: subestrutura ótima (a melhor rota total é construída a partir das melhores sub-rotas) e sobreposição de subproblemas (os mesmos caminhos parciais aparecem várias vezes no cálculo).

O subproblema seria algo assim: qual é o menor custo pra visitar um conjunto específico de pontos e terminar num ponto em particular? Com isso, o algoritmo vai construindo a solução ótima de trás pra frente, sem precisar recalcular os mesmos caminhos várias vezes.

O custo de tempo fica em $O(k^2 \times 2 \text{ elevado a } k)$ e o custo de memória em $O(k \times 2 \text{ elevado a } k)$, porque precisa guardar o resultado ótimo pra cada combinação de subconjunto e ponto final. Na prática, isso começa a ficar pesado a partir de uns 20 pedidos por entregador, e pra 30 pedidos já seria completamente inviável dentro do limite de 2 segundos que o sistema exige.

b)

A Divisão e Conquista faz mais sentido quando os subproblemas são realmente independentes, o que no caso da FastBite acontece quando os pedidos de regiões diferentes não disputam os mesmos entregadores e as rotas não precisam cruzar fronteiras entre zonas.

A ideia seria dividir a cidade em quadrantes ou zonas geográficas, resolver o problema de atribuição e roteamento dentro de cada zona de forma separada, e depois juntar os resultados. Isso reduz bastante a complexidade, já que cada zona tem menos pedidos e menos entregadores pra considerar.

O problema é que pedidos perto da fronteira entre duas zonas podem ser atribuídos de forma ineficiente, porque o algoritmo não enxerga a outra zona. Também não tem como balancear a carga entre regiões: se uma zona está sobrecarregada, entregadores de uma zona vizinha não conseguem ajudar. Isso pode causar atrasos mesmo quando existem recursos disponíveis próximos.

Questão 4

Critério	Greedy	Programação Dinâmica	Divisão e Conquista
Qualidade da solução	Média	Alta	Média
Complexidade de tempo	Baixa	Média	Média
Complexidade de espaço	Baixa	Alta	Alta
Viabilidade em tempo real ($\leq 2s$)	Alta	Baixa	Média
Escalabilidade com aumento de n	Média	Baixa	Alta
Facilidade de adaptação a mudanças	Alta	Média	Média

Olhando as três abordagens no contexto da FastBite, o método Greedy parece ser o mais adequado pra uso como solução principal. Ele não entrega a rota perfeita, mas entrega uma rota boa com velocidade de resposta que o sistema precisa. A Programação Dinâmica seria ótima se os volumes fossem menores e o tempo não fosse tão curto, mas no cenário de pico ela simplesmente não conseguiria responder dentro de 2 segundos. A Divisão e Conquista ajuda a escalar, mas perde qualidade nas fronteiras entre zonas, o que pode ser um problema sério numa cidade grande com pedidos espalhados. Usar o Greedy como base e adicionar um refinamento rápido parece a combinação mais realista pra esse tipo de sistema.

Questão 5

a) Uma heurística é basicamente uma forma de resolver um problema de maneira rápida, mesmo que a resposta não seja a melhor possível. Em vez de gastar muito tempo tentando

encontrar a solução perfeita, a heurística encontra uma solução boa o suficiente dentro do tempo disponível. No caso da FastBite, isso faz muito mais sentido do que tentar calcular a rota ótima, porque o sistema precisa tomar decisões a cada 30 segundos e qualquer atraso no cálculo já prejudica as entregas. Uma entrega feita um pouco menos eficiente é bem melhor do que uma entrega que nunca saiu porque o sistema ficou travado calculando.

b)

Uma solução real pra FastBite poderia funcionar em etapas. Primeiro, os pedidos do ciclo seriam agrupados por região da cidade, o que já reduz bastante a complexidade do problema antes de qualquer cálculo de rota.

Depois, dentro de cada região, um algoritmo greedy faria a atribuição inicial: cada pedido vai para o entregador disponível mais próximo do restaurante, levando em conta a capacidade e a prioridade do pedido (urgentes entram na frente).

Com a atribuição feita, o sistema ainda teria tempo pra tentar pequenas melhorias, como testar se trocar um pedido entre dois entregadores reduziria o tempo total. Esse refinamento local roda até o tempo acabar.

Por fim, tudo isso funciona com um limite fixo de 2 segundos. Quando esse tempo acaba, o sistema entrega a melhor solução que encontrou até ali e passa pro próximo ciclo.

c)

Vale buscar a solução ótima quando o número de pedidos e entregadores é pequeno o suficiente pra o cálculo ser rápido. Uma situação que me parece razoável dentro do delivery é o horário da madrugada: com poucos pedidos ativos e menos entregadores na rua, o sistema teria folga computacional pra calcular a rota exata. Nesse caso, o esforço extra de calcular o ótimo pode se traduzir em entregas visivelmente mais rápidas, o que melhora a experiência do cliente num horário em que a operação já é mais tranquila.

Questão 6

Quando comecei essa atividade, achei que o objetivo de qualquer sistema era sempre encontrar a resposta certa de verdade. Parecia óbvio que entregar a solução ótima seria sempre melhor. Mas aí eu fui entendendo que isso nem sempre é possível, e mais importante, nem sempre é necessário.

O problema da FastBite é NP-Completo, o que significa que nenhum algoritmo conhecido consegue resolver de forma ótima em tempo útil quando o número de pedidos e entregadores é grande. Tentar forçar uma solução perfeita nesse cenário traria o sistema por minutos, enquanto clientes ficam esperando. Isso seria pior do que uma rota levemente subótima entregue em 2 segundos.

O que aprendi é que bom o suficiente vira a melhor decisão técnica quando o custo de buscar o ótimo é maior do que o benefício que ele traria. No caso do delivery, uma rota que chega em 18 minutos usando uma heurística é muito melhor na prática do que uma rota de 17 minutos que levou 5 minutos pra ser calculada. O cliente não sente essa diferença de um minuto, mas sentiria muito se o sistema travasse.

Acho que o ponto que mais ficou comigo é que saber quando não tentar achar a solução perfeita é tão importante quanto saber como achá-la. Em sistemas que precisam funcionar em

tempo real com grande volume de dados, a habilidade de escolher a abordagem certa pro contexto certo é o que separa uma solução que funciona na prática de uma que funciona só no papel.